Interface 2026年11月号 オフ会 2026-10-16

stackchan-idf と Local LLM

stackchan-idf × XiaoZhi サーバー × ローカル LLM
× Cloudflare Tunnel

井田 健太 / @ciniml



01 自己紹介



井田 健太

X: @ciniml
GitHub: github.com/ciniml

- 組込みソフト / FPGA の人。**組込み Rust** と **MCU 向けファームウェア開発**
- ESP-IDF 製のｽﾀｯｸﾁｬﾝファームウェア **stackchan-idf** を開発中
- 著書『基礎から学ぶ 組込みRust』(C&R研究所)
  Interface 2025 年 12 月号 別冊付録 (FPGA デザイン集) 執筆

02 スタックチャンと stackchan-idf

スタックチャン



- M5Stack を頭にした**手のひらサイズの卓上ロボット**
- ししかわさん作の**オープンソース** ハードウェア

- サーボ 2 個で首を振り、画面の顔で表情を出す
- ハードが公開されているので**ファームウェアもいろいろ**
  - Arduino 版、Moddable 版、AIスタックチャン、M5Stack 公式 …

stackchan-idf

- 私が作っている**スタックチャン用ファームウェアの 1 つ**
  - ESP-IDF 5.5 / C++20 でフルスクラッチ (公式のフォークではない)
- CoreS3 / AtomS3R / StopWatch に対応
- ブラウザから書き込み・設定できる
  - Web Flasher、Bluetooth / Wi-Fi の設定ページ
- 主な機能: **AI 対話応答**、本体だけで動く音声合成、表情のカスタマイズ、LT タイマー

03 stackchan-idf の対話応答

- 機体が WebSocket で会話サーバーに直結
  - マイク音声を送り、返ってきた音声を再生・口パク
  - 接続先は **OpenAI Realtime / Gemini Live / XiaoZhi** から選択
- 会話中に LLM が**機体のツール**を呼べる
  - 体: 表情・首振り・カタコト声
  - お手伝い: 時刻・状態・記憶・タイマー
  - `ask_agent` : 調べ物を家の Claude Code に頼む



> これまでは OpenAI / Gemini の**クラウド頼み** → 今回、XiaoZhi で**自宅のサーバー**につないだ

04 XiaoZhi (小智) — 自宅に立てられる会話サーバー

もとは中国の「小智」用

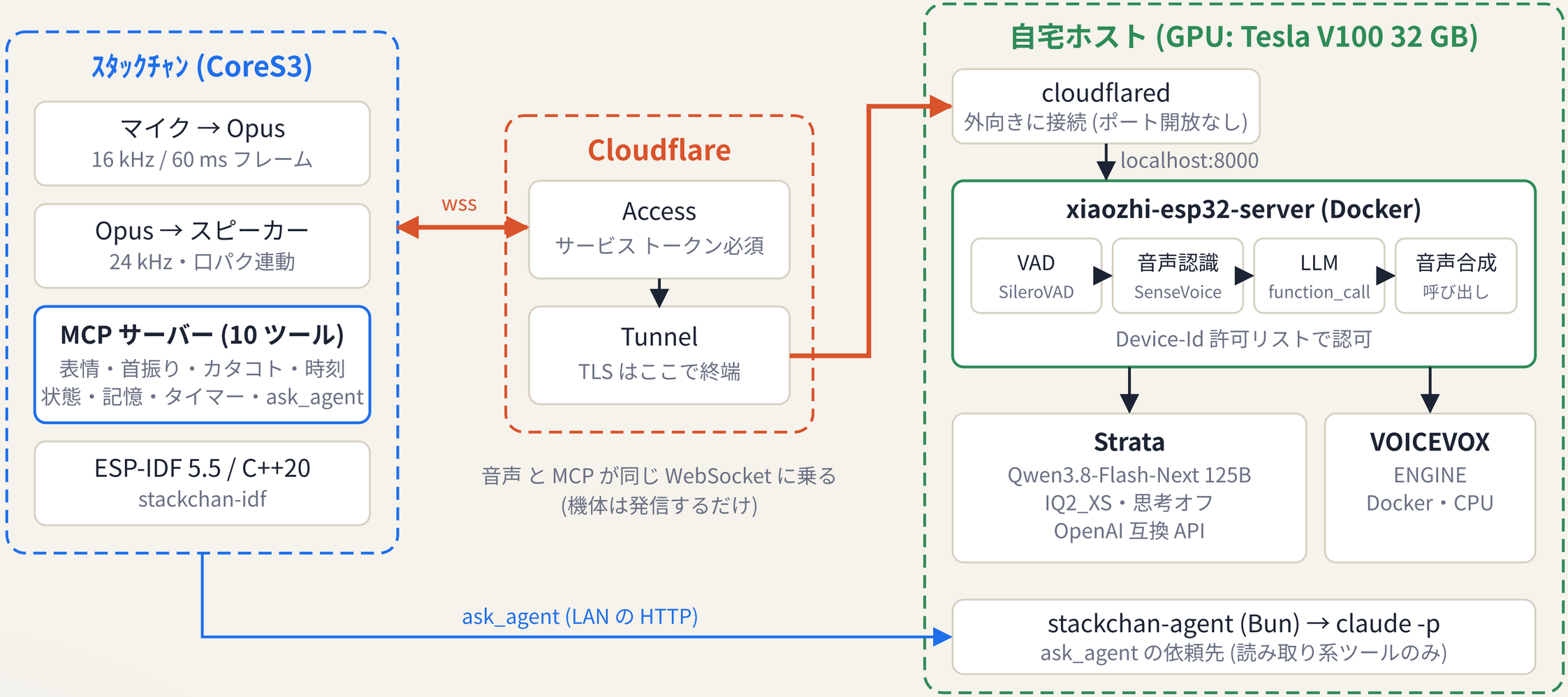
- **xiaozhi-esp32**: 中国発のオープンソース AI 音声対話デバイス
  - ESP32 にマイクとスピーカーを付けて話しかける
- 作者が**公式のクラウド サービス**を運営
- 機体とサーバーの間は**公開された通信プロトコル**
  - WebSocket + Opus 音声 + JSON、MCP でツール呼び出し

サーバーは自分で立てられる

- コミュニティ製の `xiaozhi-esp32-server`
  - Docker 構成と**一発セットアップ用スクリプト**が公開済み
  - サーバー本体は 2 コア / 4 GB 程度から (LLM は別)
- VAD / 音声認識 / LLM / 音声合成を**部品ごとに選べる**
  - クラウド API でも**ローカル**でも

stackchan-idf は XiaoZhi クライアントを実装済み → **接続先 URL を自宅サーバーにするだけ**

05 全体構成


ｽﾀｯｸﾁｬﾝ (CoreS3)
マイク → Opus
16 kHz / 60 ms フレーム
Opus → スピーカー
24 kHz・口パク連動
MCP サーバー (10 ツール)
表情・首振り・カタコト・時刻
状態・記憶・タイマー・ask_agent
ESP-IDF 5.5 / C++20
stackchan-idf
WSS
Cloudflare
Access
サービス トークン必須
Tunnel
TLS はここで終端
音声 と MCP が同じ WebSocket に乗る
(機体は発信するだけ)
自宅ホスト (GPU: Tesla V100 32 GB)
cloudflared
外向きに接続 (ポート開放なし)
localhost:8000
xiaozhi-esp32-server (Docker)
VAD
SileroVAD
音声認識
SenseVoice
LLM
function_call
音声合成
呼び出し
Device-Id 許可リストで認可
Strata
Qwen3.8-Flash-Next 125B
IQ2_XS・思考オフ
OpenAI 互換 API
VOICEVOX
ENGINE
Docker・CPU
ask_agent (LAN の HTTP)
stackchan-agent (Bun) → claude -p
ask_agent の依頼先 (読み取り系ツールのみ)

06 ホスト側の部品

役割	使ったもの	メモ
会話サーバー	**xiaozhi-esp32-server** (Docker)	フォークして日本語向けに修正
VAD	SileroVAD	発話区間の切り出し
音声認識	FunASR **SenseVoiceSmall**	言語は日本語に固定
LLM	**Strata** + Qwen3.8-Flash-Next 125B (IQ2_XS)	V100 32 GB 1 枚。OpenAI 互換 API
音声合成	**VOICEVOX ENGINE** (CPU)	プロバイダを自作して追加
委譲	stackchan-agent → `claude -p`	ask_agent 用 (ここだけクラウド)

> **そこそこの計算資源** (GPU 1 枚 + CPU) で、会話の頭が丸ごと**ローカル**で動く。音声合成は CPU で十分

07 家の外から機体をつなぐ

課題 機体を持ち歩いても家のサーバーにつなぎたい

ポート開放はしたくない。ESP32 に重い仕組みは載せたくない

対策 Cloudflare Tunnel で WebSocket を 1 本だけ公開

ホストの cloudflared が外向きに接続。TLS は Cloudflare 側

- 認可は 2 段
  - ① Cloudflare Access: 機体が**追加ヘッダー**でサービス トークンを送る
  - ② XiaoZhi サーバー: **Device-Id 許可リスト** (知らない機体は即切断)
- LLM・音声合成・管理用 HTTP は**非公開**のまま

ESP32 側の落とし穴

wss にすると TLS で**内蔵 RAM が枯渇**
(会話中の空き 11 KiB)

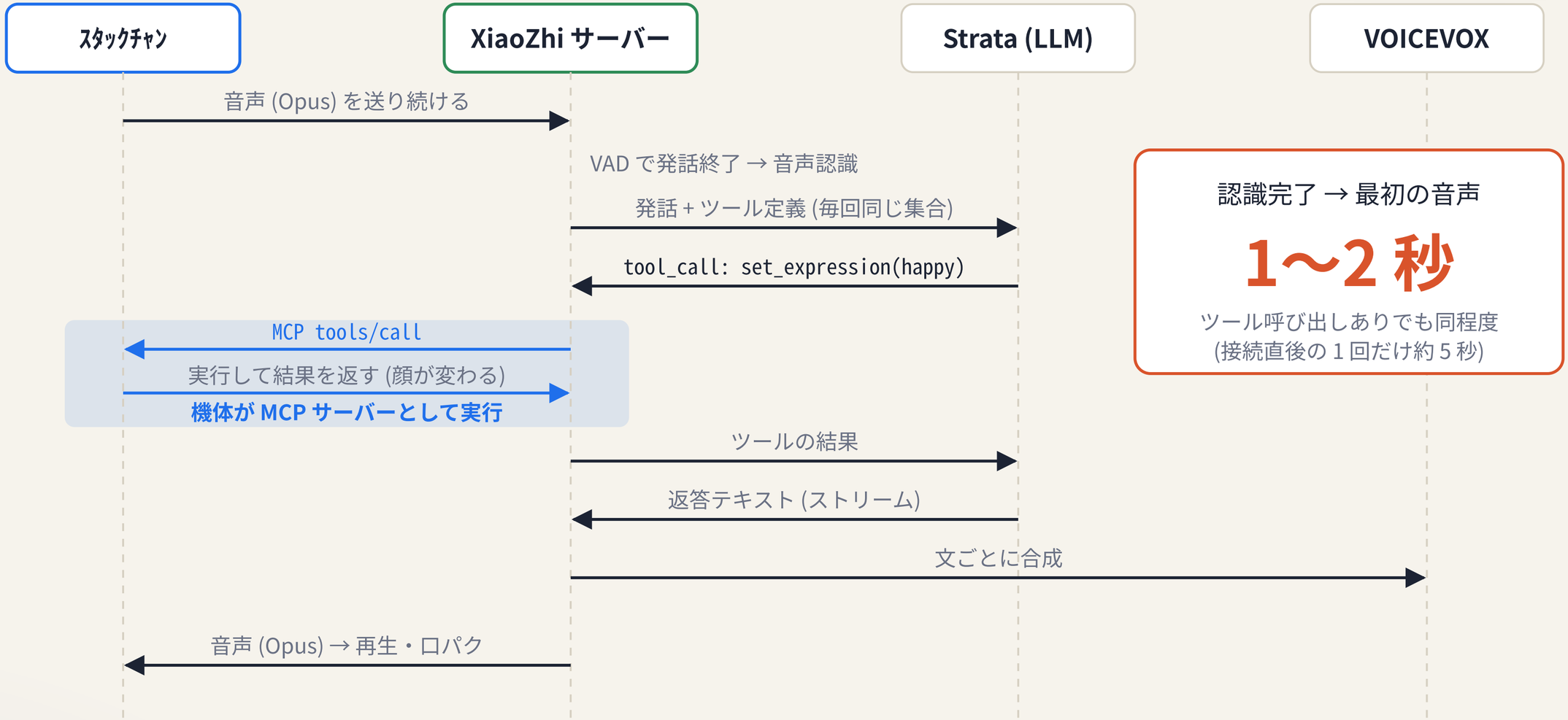
対策

Opus デコードを **PSRAM にスタックを置いた専用タスク**へ移動
WebSocket タスク 16 → 6 KiB
空き 11 → **27 KiB**

LAN では直結も可

同じ設定項目で `ws://` の LAN アドレスに切り替えるだけ

08 1 ターンの流れ — 機体が MCP サーバー



09 ハマりどころ (ローカル LLM × XiaoZhi サーバー)

応答に `\u30b9\u30bf…` が混ざって読み上げられる

認識結果が JSON のまま LLM に渡り、Qwen が返事までエスケープ → 素のテキストで渡す

雑談中に突然「バイバイ」して切断

組み込みの退出判定ツールを誤発火 → 必須ツールから外す

ツールを呼んだ後から中国語で話し出す

ベース プロンプトが中国語 → 日本語に訳して差し替え

「さっきの物語の話だけど…」と蒸し返す

毎接続、few-shot 例の会話が履歴に入っていた → 短い挨拶に差し替え

速度: キャッシュを効かせる

2 回目の呼び出しでツール集合が変わるとプロンプト キャッシュが外れて **3〜5 秒** → 固定して **0.2〜1.8 秒**

速度: 思考をオフ

会話に長考は要らない。最初のトークンまで **約 0.3 秒**

デバッグ

サーバーと LLM の間に**記録プロキシ**を挟み、実際のプロンプトを見るのが早い

10 まとめ

- ｽﾀｯｸﾁｬﾝの会話の頭を**自宅のそこそこの計算資源**に移せた
  - XiaoZhi サーバー + Strata (125B) + VOICEVOX
  - 応答は 1〜2 秒。機体のツールも MCP で呼べる
- 家の外からは **Cloudflare Tunnel + Access** で安全に
  - 機体は発信するだけ。ポート開放なし
- これから
  - 割り込み (barge-in) と相槌の改善
  - ask_agent の依頼先もローカル LLM に

リポジトリ
github.com/ciniml/stackchan-idf

Web Flasher
ciniml.github.io/stackchan-idf/

井田 健太 / @ciniml